



广州应用科技学院

操作系统课程设计报告

(第 1 组)

题目： 修改内核及 Linux 内核重构

学生学号： 202410004688

学生姓名： 叶志挺

专业班级： 22 计算机科学与技术 11 班

老师姓名： 黄天信

2024 年 11 月 29 日

目录

1 概述	1
1.1 内核修改内容介绍	2
1.2 内核编译环境介绍	2
2 内核的编译和修改	3
2.1 从网上下载内核源代码	3
2.2 扩容虚拟机内存	4
2.3 解压安装内核源文件	8
2.4 安装必要软件	9
2.5 编译环境处理	10
2.6 配置编译参数	11
2.7 依赖关系	12
2.8 编译二进制内核映像文件	13
2.9 编译模块并且安装模块	14
2.10 拷贝新内核到引导区	14
2.11 安装启动管理器	15
3 内核重编译	17
3.1 Linux 内核源代码下载及其版本说明	17
3.2 在 Linux 内核源代码中添加 systest () 系统调用	17
4 验证 systest () 系统调用	25
4.1 编译验证程序 test.c	25
4.2 运行验证程序 test	25
5 课程设计的体会	27
5.1 个人工作体会	27
5.2 团队合作	32

修改内核及 Linux 内核重编译

——为 Linux 内核增加系统调用 `sys_test()`

1 概述

本课程设计聚焦于 Linux 系统内核相关操作，旨在通过动手操作提升对操作系统内核结构、功能和编译过程的理解。具有明确的任务和目的，同时在内容、重点及过程方面有相应的过程，最终指向完整的系统修改与验证流程。

课程设计任务和目的

技能训练	对 Linux 系统内核修改及系统重编译的能力，建立软件设计观念。深入理解操作系统底层工作机制，提升系统级编程与优化能力，为未来从事系统开发、性能优化等工作奠定基础。
功能实现与系统重组	学会深入操作系统内核源代码，进行功能修改与添加，如按照要求添加特定系统调用函数。随后通过重编译源代码，掌握将修改后的代码重组为正确、可运行操作系统的全过程，确保系统在功能扩展后的稳定性与正确性。

课程设计重点和难点

内容	围绕为操作系统内核添加系统调用展开，添加函数，掌握内核功能扩展的基本流程与方法，聚焦于关键技术环节。
重点把握	将系统重编译及系统重组过程、方法以及过程中问题的解决作为重点。熟悉编译工具与参数配置，深入理解系统各组件间的关系，遇到如链接错误、配置冲突等问题并解决。

课程设计过程

环境准备与初步编译	下载内核版本（ 2.6.33.3）并解压，进行首次编译。此过程中遭遇存储空间不足、数据源连接问题或其他未知问题，分别对应扩容、换源及网上查询解决方案等方法。
内核修改与系统重构	掌握初步编译后，着手修改内核源代码，添加系统调用，接着对改后系统重编译重构，最终实现新系统启动。
功能验证	通过编译验证程序，对新添加的系统调用函数进行严格测试，确保其功能的正确性与稳定性。

1.1 内核修改内容介绍

为操作系统添加一个系统调用函数:sys_str_process()

输入参数:hello, word

返回结果:HELLO, WORD

该系统调用的功能是大小写转化，并相应地返回 0 或 1。具体步骤：

1. 在系统调用表中添加新表项，以便内核识别和处理新的系统调用。
2. 增加新的系统调用号，确保系统调用的唯一性和正确性。
3. 实现 sys_str_process() 函数的实体，编写相应的逻辑断。
4. 编写测试程序，用于验证新系统调用的正确性和功能性。

1.2 内核编译环境介绍

处理器	i5-12500
内存	16g
操作系统	windows10, VMware, ubuntu9. 10
ubuntu9. 10 环境	内存 2G，处理器 2 个，硬盘：32GB

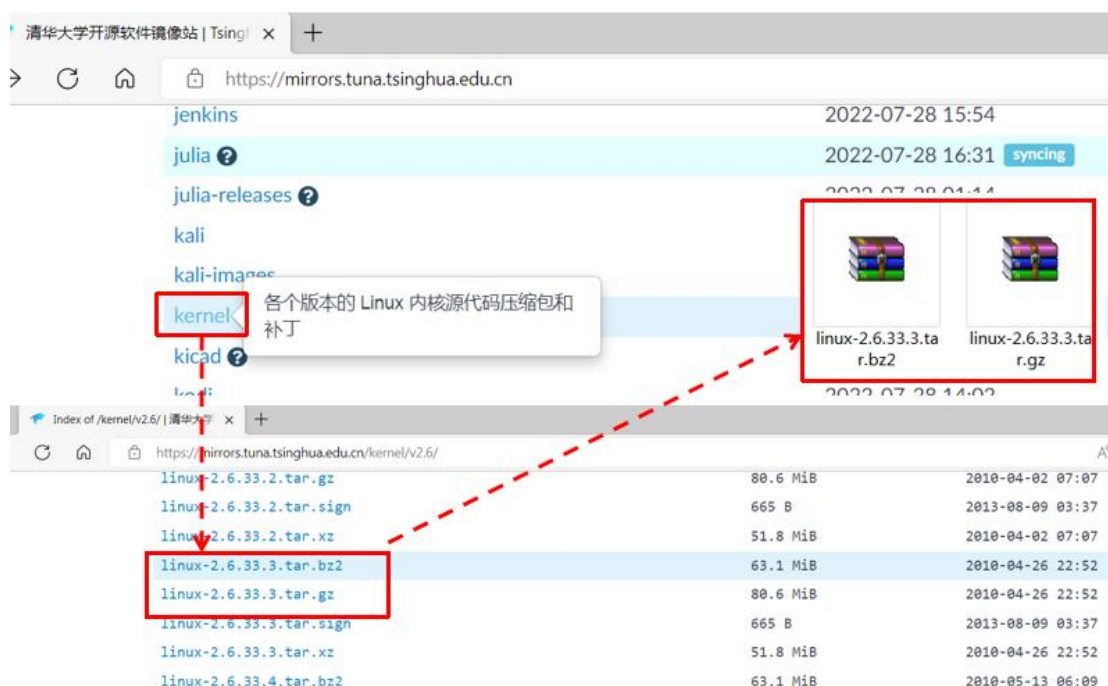
2 内核的编译和修改

2.1 从网上下载内核源代码

在本次课程设计中，获取 ubuntu 内核源代码是关键步骤之一。

清华镜像站 (<https://mirrors.tuna.tsinghua.edu.cn>)

内核版本：2.6.33.3 版本的内核



下载两个文件：

linux-2.6.33.3.tarbz2

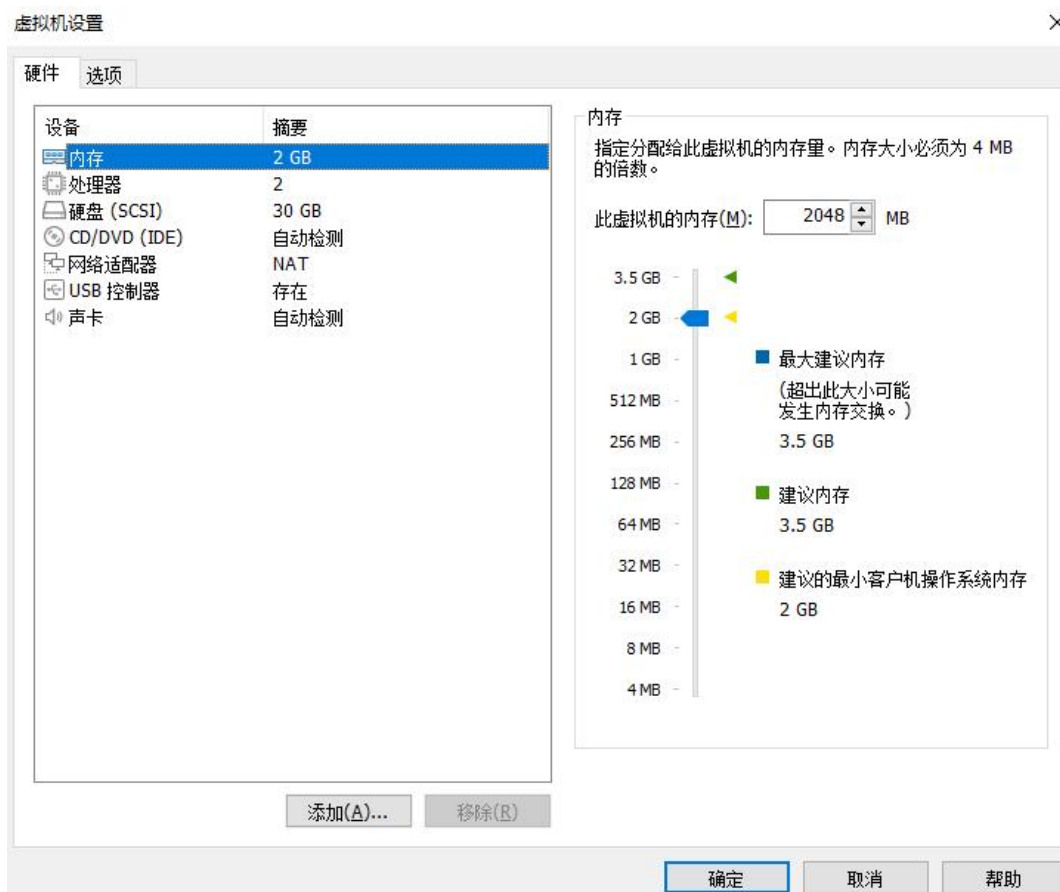
linux-2.6.33.3.tar.gz



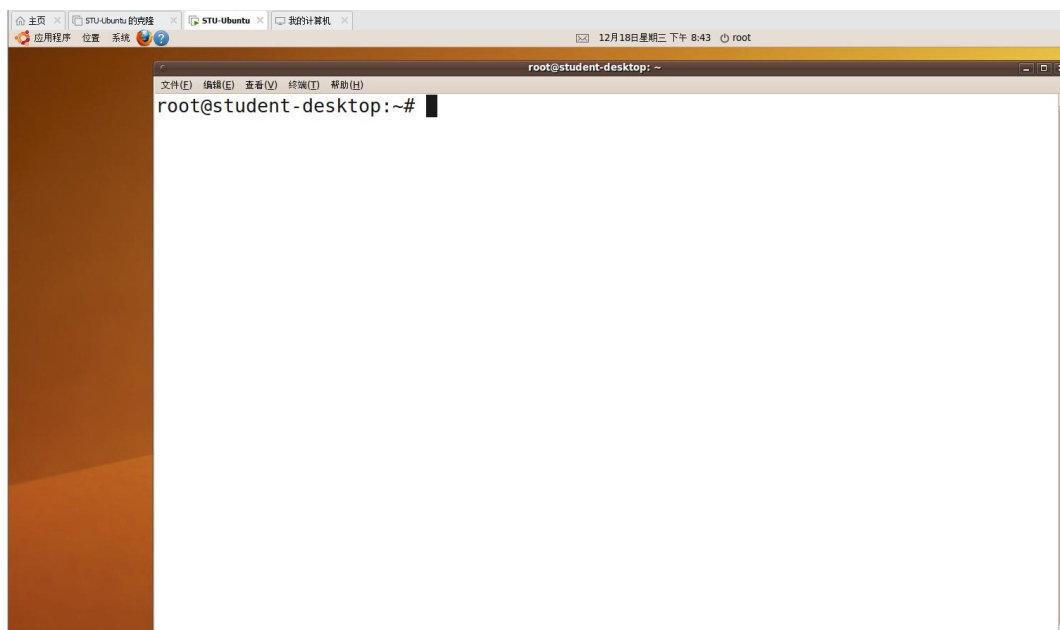
以上就是 Linux2.6.33.3 内核源代码。在真正解压前，需要先扩容文件系统(ext4)的容量。

2.2 扩容虚拟机内存

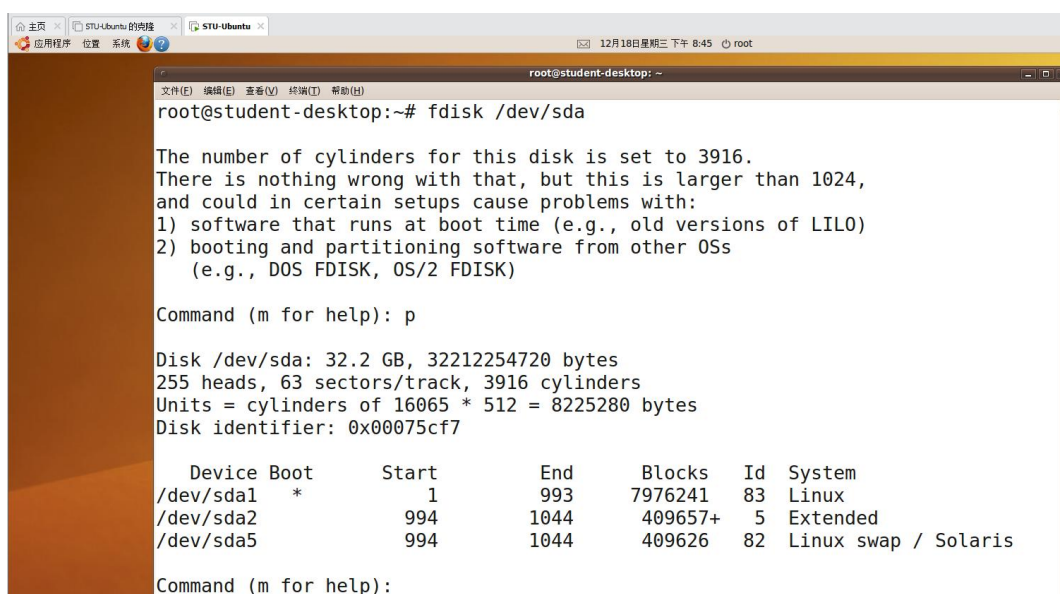
1. 把原虚拟机的内存 1GB 和硬盘 8GB 改成内存 2GB 和硬盘 32GB



2. 然后启动虚拟机，用 root 账号登录后打开终端



3. 用命令“fdisk /dev/sda”启动 fdisk



3.1 先用 d 命令删除所有的分区,再用 n 命令创建两个主分区 sda1 (30GB, Linux 分区标识码 0x83) 和 sda2 (剩余的所有空间, swap 分区)

```
Command (m for help): d
Partition number (1-5): 1

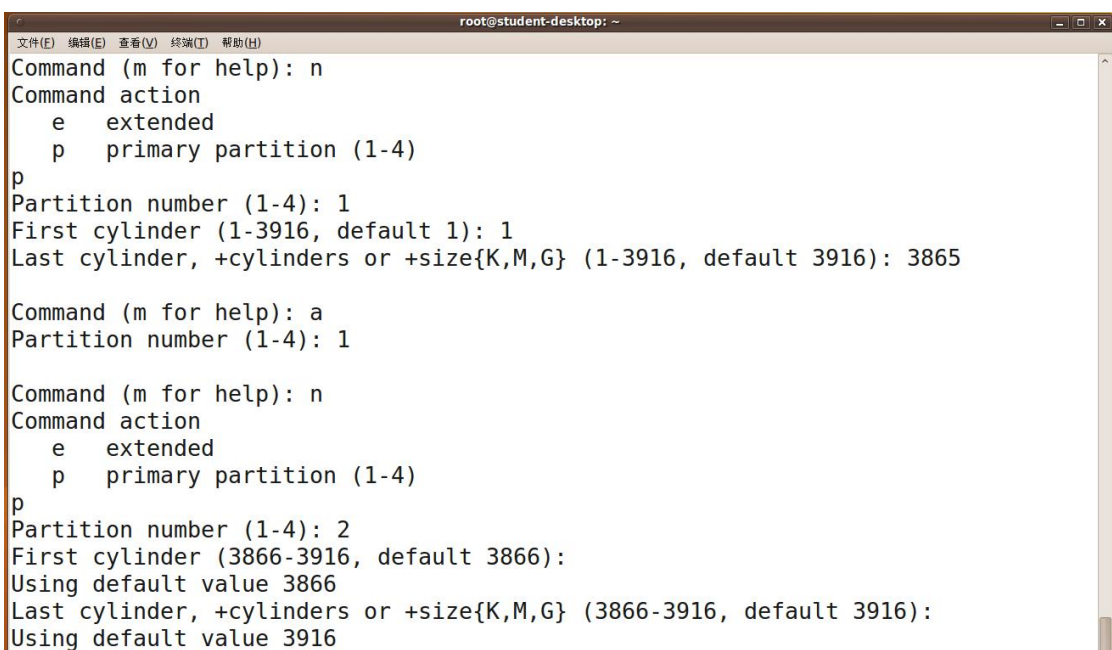
Command (m for help): d
Partition number (1-5): 2

Command (m for help): p

Disk /dev/sda: 32.2 GB, 32212254720 bytes
255 heads, 63 sectors/track, 3916 cylinders
Units = cylinders of 16065 * 512 = 8225280 bytes
Disk identifier: 0x00075cf7
```

Device	Boot	Start	End	Blocks	Id	System
--------	------	-------	-----	--------	----	--------

```
Command (m for help):
```



```
root@student-desktop: ~
文件(F) 编辑(E) 查看(V) 终端(T) 帮助(H)
Command (m for help): n
Command action
  e   extended
  p   primary partition (1-4)
p
Partition number (1-4): 1
First cylinder (1-3916, default 1): 1
Last cylinder, +cylinders or +size{K,M,G} (1-3916, default 3916): 3865

Command (m for help): a
Partition number (1-4): 1

Command (m for help): n
Command action
  e   extended
  p   primary partition (1-4)
p
Partition number (1-4): 2
First cylinder (3866-3916, default 3866):
Using default value 3866
Last cylinder, +cylinders or +size{K,M,G} (3866-3916, default 3916):
Using default value 3916
```

3.2 用 `a` 命令将 `sda1` 设置为可启动分区，用 `t` 命令将 `sda2` 设置为 `swap` 分区（标识码：0x82）

```

root@student-desktop: ~
Disk /dev/sda: 32.2 GB, 32212254720 bytes
255 heads, 63 sectors/track, 3916 cylinders
Units = cylinders of 16065 * 512 = 8225280 bytes
Disk identifier: 0x00075cf7

   Device Boot      Start         End      Blocks   Id  System
/dev/sda1  *           1         3865     31045581   83   Linux
/dev/sda2           3866         3916      409657+    83   Linux

Command (m for help): t
Partition number (1-4): 2
Hex code (type L to list codes): 82
Changed system type of partition 2 to 82 (Linux swap / Solaris)

Command (m for help): p

Disk /dev/sda: 32.2 GB, 32212254720 bytes
255 heads, 63 sectors/track, 3916 cylinders
Units = cylinders of 16065 * 512 = 8225280 bytes
Disk identifier: 0x00075cf7

   Device Boot      Start         End      Blocks   Id  System
/dev/sda1  *           1         3865     31045581   83   Linux
/dev/sda2           3866         3916      409657+    82   Linux swap / Solaris

```

3.3 用 w 命令存盘后退出。

```

Command (m for help): w
The partition table has been altered!

Calling ioctl() to re-read partition table.

WARNING: Re-reading the partition table failed with error 16: Device or resource busy.
The kernel still uses the old table. The new table will be used at
the next reboot or after you run partprobe(8) or kpartx(8)
Syncing disks.
root@student-desktop:~# █

```

4. 重新打开虚拟机，然后重新打开终端

5. 执行命令：resize2fs /dev/sda1 完成虚拟机扩容

```

root@student-desktop:~# resize2fs /dev/sda1
resize2fs 1.41.9 (22-Aug-2009)
The filesystem is already 7761395 blocks long.  Nothing to do!

root@student-desktop:~# df -TH
文件系统      类型      容量  已用  可用 已用% 挂载点
/dev/sda1     ext4       32G   8.6G   22G   29% /
udev          tmpfs      1.1G   226k   1.1G    1% /dev
none          tmpfs      1.1G   1.1M   1.1G    1% /dev/shm
none          tmpfs      1.1G  103k   1.1G    1% /var/run
none          tmpfs      1.1G    0     1.1G    0% /var/lock
none          tmpfs      1.1G    0     1.1G    0% /lib/init/rw

```



2.3 解压安装内核源文件

把压缩文件拷贝在/usr/src 目录下



解压流程:

1. 进入 /usr/src 目录

```
cd /usr/src
```

2. 解压源文件

```
bzip2 -d linux-2.6.33.3.tar.bz2
```

```
tar -xvf linux-2.6.33.3.tar
```

此时用 `dir` 命令查看当前目录下，增加了一个“linux-2.6.33.3”的子目录里面存放了刚才解压出来所有的源

文件。然后把目录链接改成成“linux”目录。

In -s linux-2.6.33.3 linux

2.4 安装必要软件

1. 安装必要软件前提换源

先进入源文件所在目录

```
cd /etc/apt
```

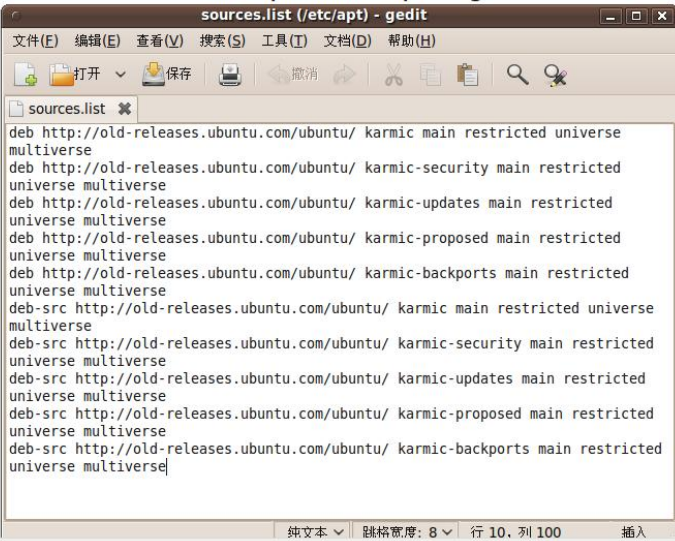
备份源来的数据源

```
cp sources.list source.list.old
```

打开编辑文件 sources

```
gedit sources.list
```

```
root@student-desktop:/etc/apt# cp sources.list source.list.old
root@student-desktop:/etc/apt# gedit sources.list
```



完成换源操作

```
apt-get update
```

2. 安装必要工具

```
sudo apt-get install build-essential kernel-package libncurses5-dev fakeroot
```

```
sudo aptitude install libgt3-headers libgt3-mt-dev libgt3-compat-headers  
libgt3-mt
```

2.5 编译环境处理

1. 清理以前编译残留

清理以前编译残留

```
cd /usr/src/linux
```

```
make mrproper
```

```
root@student-desktop:/etc/apt# cd /usr/src/linux
root@student-desktop:/usr/src/linux# make mrproper
CLEAN    arch/x86/boot/compressed
CLEAN    arch/x86/boot
CLEAN    /usr/src/linux-2.6.33.3
CLEAN    arch/x86/kernel/acpi/realmode
CLEAN    arch/x86/kernel/cpu
CLEAN    arch/x86/kernel
CLEAN    arch/x86/vdso
CLEAN    arch/x86/lib
CLEAN    drivers/char
CLEAN    drivers/eisa
CLEAN    drivers/gpu/drm/radeon
CLEAN    drivers/md
CLEAN    drivers/net/wan
CLEAN    drivers/scsi/aic7xxx
CLEAN    drivers/scsi
CLEAN    firmware
```

2. 调整链接 asm、linux 和 scsi 的正确指向

调整链接 asm、linux 和 scsi 的正确指向

```
cd /usr/include/
```

```
rm -r asm linux scsi
```

```
ln -s /usr/src/linux/include/asm-generic asm
```

```
ln -s /usr/src/linux/include/linux linux
```

```
ln -s /usr/src/linux/include/scsi scsi
```

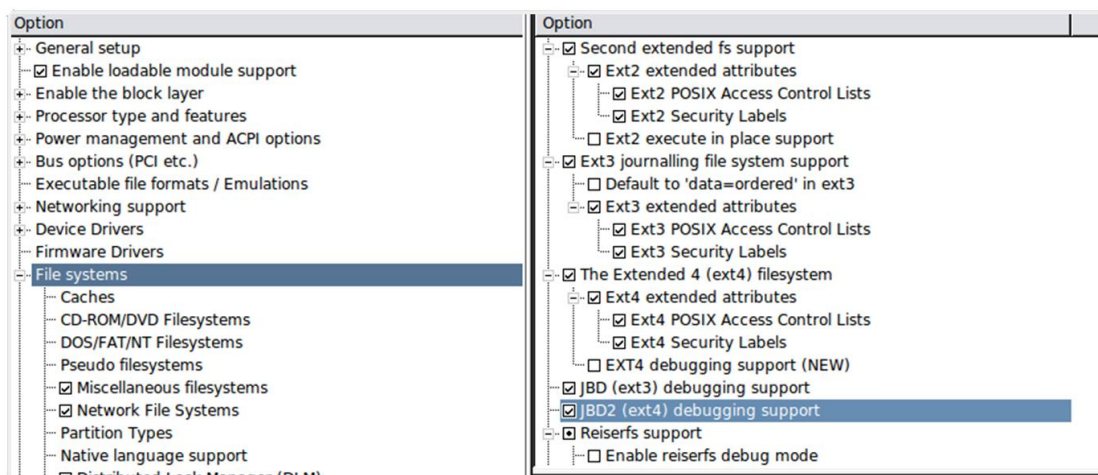
```
root@student-desktop:/usr/src/linux# cd /usr/include/
root@student-desktop:/usr/include# rm -r asm linux scsi
root@student-desktop:/usr/include# ln -s /usr/src/linux/include/asm-generic asm
ln -s: command not found
root@student-desktop:/usr/include# ln -s /usr/src/linux/include/asm-generic asm
root@student-desktop:/usr/include# ln -s /usr/src/linux/include/linux linux
root@student-desktop:/usr/include# ln -s /usr/src/linux/include/scsi scsi
```

2.6 配置编译参数

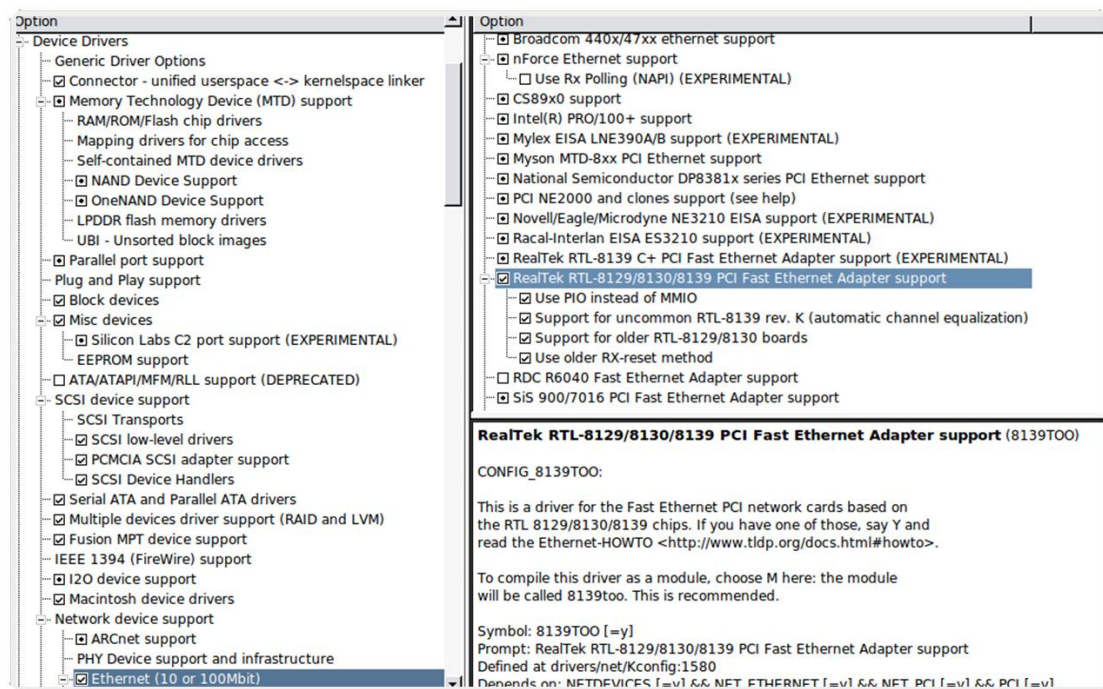
配置编译参数

```
cd /usr/src/linux
```

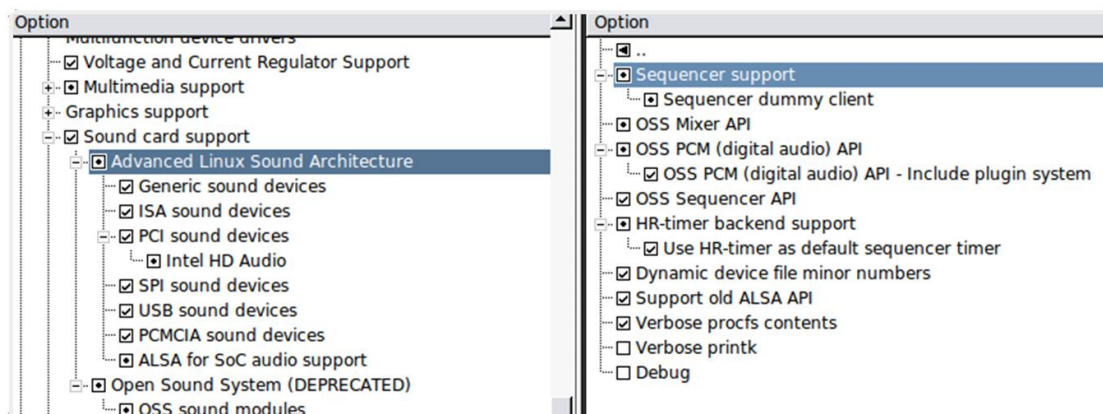
make xconfig 使用图形命令进入图形界面



图形界面下的文件系统设置



图形界面下的网卡驱动设置



图形界面下的声卡驱动设置

2.7 依赖关系

1. 检查并决定是否生成依赖

检查并决定是否生成依赖

make dep

```
root@student-desktop:/usr/src/linux# make dep
HOSTLD scripts/kconfig/conf
scripts/kconfig/conf -s arch/x86/Kconfig
*** Warning: make dep is unnecessary now.
root@student-desktop:/usr/src/linux#
```

2. 清除旧编译数据

清除旧编译数据

```
make clean
```

```
root@student-desktop:/usr/src/linux# make clean
root@student-desktop:/usr/src/linux#
```

2.8 编译二进制内核映像文件

编译二进制内核映像文件

```
cd /usr/src/linux
```

```
make bzImage
```



2.9 编译模块并且安装模块

编译模块并且安装模块

```
cd /usr/src/linux
```

```
make -j 2 modules
```

```
root@student-desktop:/usr/src/linux# make -j 2 modules
CHK      include/linux/version.h
CHK      include/generated/utsrelease.h
CALL     scripts/checksyscalls.sh
AS [M]   arch/x86/crypto/aes-i586-asm_32.o
CC [M]   arch/x86/crypto/aes_glue.o
AS [M]   arch/x86/crypto/salsa20-i586-asm_32.o
CC [M]   arch/x86/crypto/salsa20_glue.o
AS [M]   arch/x86/crypto/twofish-i586-asm_32.o
CC [M]   arch/x86/crypto/twofish_glue.o
```

```
make -j 2 modules install
```



2.10 拷贝新内核到引导区

1. 拷贝新内核到引导区

拷贝新内核到引导区

```
cp /usr/src/linux/System.map /boot/System.map-2.6.33.3
cp /usr/src/linux/arch/i386/boot/bzImage /boot/vmlinuz-2.6.33.3
```

```
cp /usr/src/linux/.config /boot/config-2.6.33.3
```

```
root@student-desktop:/usr/src/linux# cp /usr/src/linux/System.map /boot/System.map-2.6.33.3
root@student-desktop:/usr/src/linux# cp /usr/src/linux/arch/i386/boot/bzImage /boot/vmlinuz-2.6.33.3
root@student-desktop:/usr/src/linux# cp /usr/src/linux/.config /boot/config-2.6.33.3
root@student-desktop:/usr/src/linux#
```

2. 建立要载入的 ramdisk 映像文件

建立要载入的 ramdisk 映像文件

```
mkinitramfs -o /boot/initrd.img-2.6.33.3 2.6.33.3
```

3. 重新创建新的链接

拷贝新内核到引导区

```
cd /boot

rm -f System.map vmlinuz

ln -s vmlinuz-2.6.33.3 vmlinuz

ln -s System.map-2.6.33.3 System.map
```

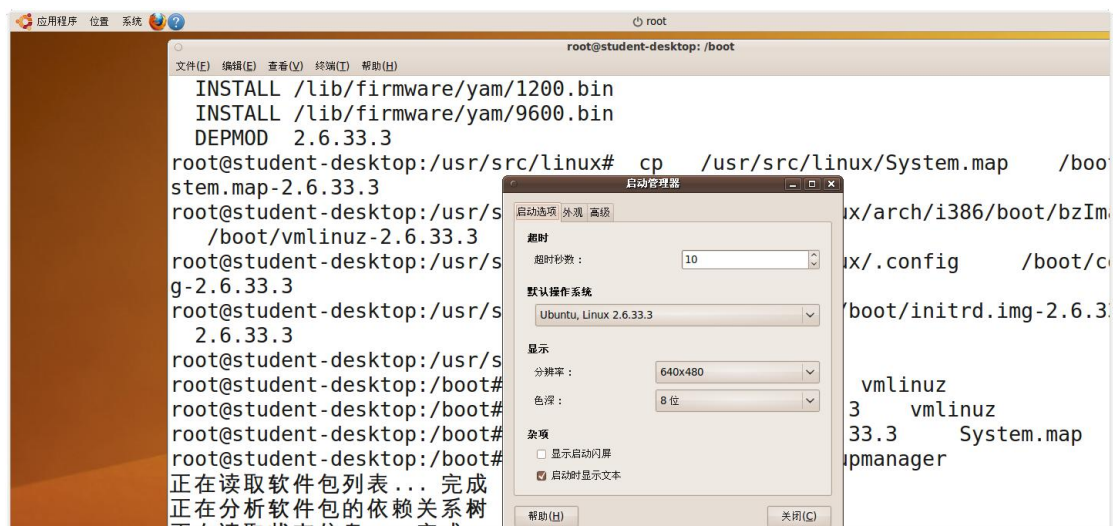
2.11 安装启动管理器

1. 安装启动管理器

```
sudo apt-get install startupmanager
```

2. 打开虚拟机，并点击启动管理器

重新启动虚拟机，并打开系统栏—>系统管理—>启动管理器即可



3、关机并重新开机

关机。

重新开机，稍作等待。

顺利开机成功，登录后，用“uname-r”查看内核版本号，完全更新成了新内核 2.6.33.3 版本！

```
root@student-desktop:~# uname -r
2.6.33.3
```

3 内核重编译

3.1 Linux 内核源代码下载及其版本说明

名称	修改日期	类型	大小
 linux-2.6.33.3	2024/12/4 19:44	WinRAR 压缩文件	385,880 KB
 linux-2.6.33.3.tar	2024/12/11 19:50	WinRAR 压缩文件	64,648 KB
 linux-2.6.33.3.tar	2024/12/11 19:50	WinRAR 压缩文件	82,569 KB

Linux 内核源代码下载及其版本说明	
清华站镜像	https://mirrors.tuna.tsinghua.edu.cn
内核版本	2.6.33.3

3.2 在 Linux 内核源代码中添加系统调用

1. 修改系统调用表项

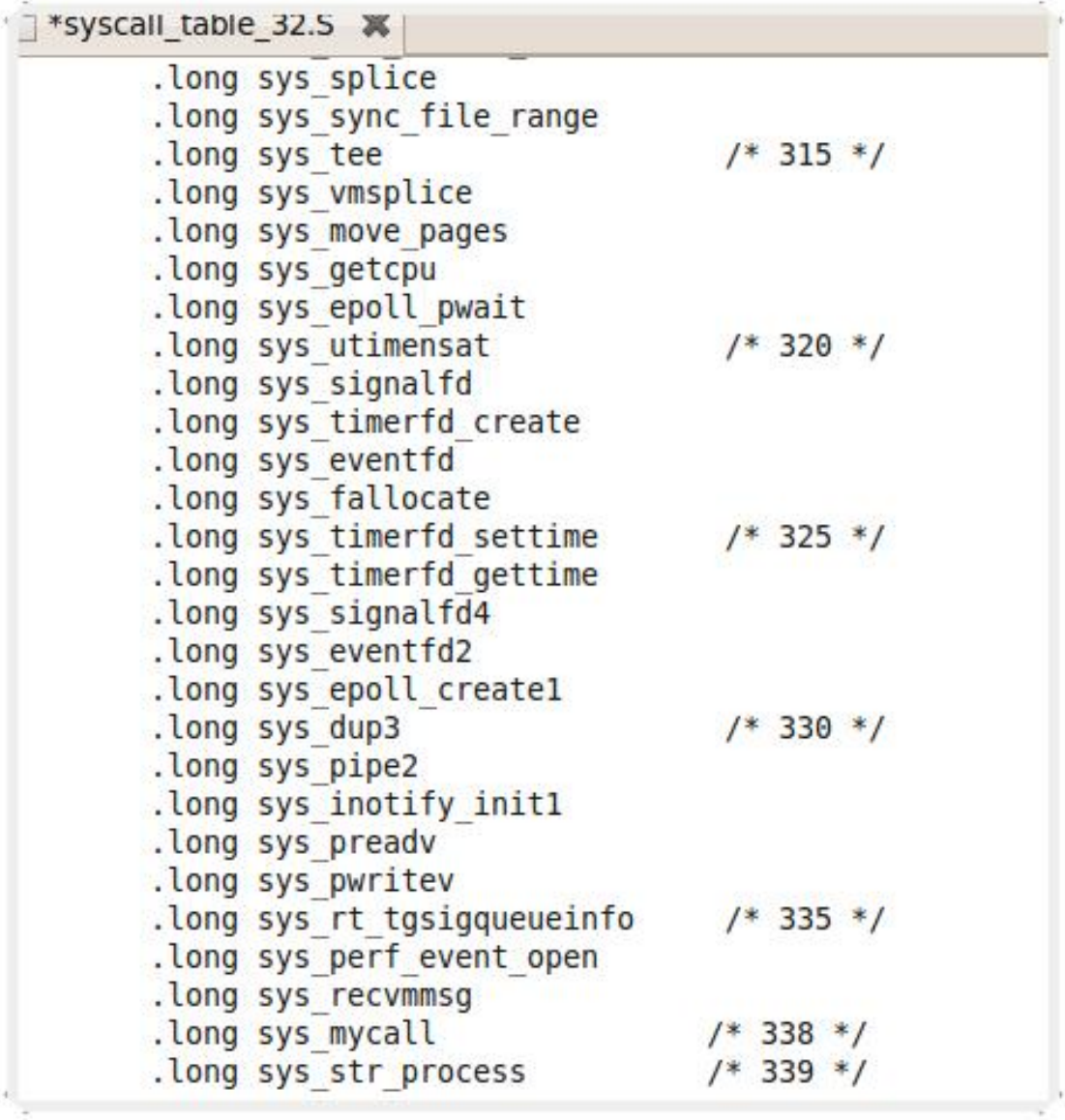
我们需要将这个函数添加到系统调用表中，以便操作系统能够识别并调用它。系统调用表是操作系统内核中一个关键的数据结构，它存储了所有可用系统调用的入口地址。

在文件 “syscall_table_32.S”（如图 1 所示）中添加新的系统调用表项。此文件定义了系统调用的入口地址，新的系统调用需要按照顺序添加到表的末尾。添加 “long sys_str_process” 作为新的系统调用入口。

在/usr/src/linux/arch//x86/kernel/syscall_table_32.S 最后增加一个系统表项:

增加表项

```
.long sys_str_process /* 339 */
```



The screenshot shows a text editor window titled '*syscall_table_32.S'. The file contains a list of system call definitions, each consisting of a '.long' instruction followed by a symbol name and a comment. The symbols listed are: sys_splice, sys_sync_file_range, sys_tee, sys_vmsplice, sys_move_pages, sys_getcpu, sys_epoll_pwait, sys_utimensat, sys_signalfd, sys_timerfd_create, sys_eventfd, sys_fallocate, sys_timerfd_settime, sys_timerfd_gettime, sys_signalfd4, sys_eventfd2, sys_epoll_create1, sys_dup3, sys_pipe2, sys_inotify_init1, sys_preadv, sys_pwritev, sys_rt_tgsigqueueinfo, sys_perf_event_open, sys_recvmmsg, sys_mycall, and sys_str_process. Comments like /* 315 */, /* 320 */, /* 325 */, /* 330 */, /* 335 */, /* 338 */, and /* 339 */ are present for several entries.

```
*syscall_table_32.S
.long sys_splice
.long sys_sync_file_range
.long sys_tee /* 315 */
.long sys_vmsplice
.long sys_move_pages
.long sys_getcpu
.long sys_epoll_pwait
.long sys_utimensat /* 320 */
.long sys_signalfd
.long sys_timerfd_create
.long sys_eventfd
.long sys_fallocate
.long sys_timerfd_settime /* 325 */
.long sys_timerfd_gettime
.long sys_signalfd4
.long sys_eventfd2
.long sys_epoll_create1
.long sys_dup3 /* 330 */
.long sys_pipe2
.long sys_inotify_init1
.long sys_preadv
.long sys_pwritev
.long sys_rt_tgsigqueueinfo /* 335 */
.long sys_perf_event_open
.long sys_recvmmsg
.long sys_mycall /* 338 */
.long sys_str_process /* 339 */
```

这行代码将我们的函数添加到系统调用表中，并且为其分配了一个系统调用号 339。系统调用号是操作系统用来唯一标识每个系统调用的数字，它在系统调用的调用过程中起到了关键作用。

2. 分配系统调用号

接下来，我们还需要在

/usr/src/linux/include/asm-generic/unistd.h 文件

在 “unistd.h” 文件（如图 2 所示）中分配系统调用号。定义新的系统调用号，“#define __NR_sys_str_process 339”，并通过 “SYSCALL (__NR_sys_str_process, sys_str_process)” 将系统调用号与函数关联起来。

在/usr/src/linux/include/asm-generic/unistd.h 中添加：

630 行左右

添加系统调用号
<pre>#define __NR_str_procesl 339 __SYSCALL(__NR_str_procesl, sys_str_procesl)</pre>

```
__SYSCALL(__NR_mbind, sys_mbind)
#define __NR_get_mempolicy 236
__SYSCALL(__NR_get_mempolicy, sys_get_mempolicy)
#define __NR_set_mempolicy 237
__SYSCALL(__NR_set_mempolicy, sys_set_mempolicy)
#define __NR_migrate_pages 238
__SYSCALL(__NR_migrate_pages, sys_migrate_pages)
#define __NR_move_pages 239
__SYSCALL(__NR_move_pages, sys_move_pages)
#endif

#define __NR_rt_tgsigqueueinfo 240
__SYSCALL(__NR_rt_tgsigqueueinfo, sys_rt_tgsigqueueinfo)
#define __NR_perf_event_open 241
__SYSCALL(__NR_perf_event_open, sys_perf_event_open)
#define __NR_accept4 242
__SYSCALL(__NR_accept4, sys_accept4)
#define __NR_recvmmsg 243
__SYSCALL(__NR_recvmmsg, sys_recvmmsg)
#define __NR_mycall 338
__SYSCALL(__NR_mycall, sys_mycall)

#define __NR_str_process 339
__SYSCALL(__NR_str_process, sys_str_process)
```

3. 实现系统调用函数实体

在“sys.c”文件（如图所示）中实现函数实体。

为Linux内核增加一个系统调用,编写这个系统调用的具体实现函数。使用gedit编辑器打开/usr/src/linux/kernel/sys.c文件

在/usr/src/linux/kernel/sys.c中添加处理函数：（添加系统调用的处理函数：大小写转换）

```
include <linux/kernel.h>
include <linux/syscalls.h>
include <linux/string.h>

/ 新系统调用函数定义, 命名为sys_str_process, 可按需更改
asm linkage long sys_str_process(const char __user *input_str, char __user *output_str, int max_length)

{
    char *temp_buf;
    int i;
    if (max_length <= 0) {
        return -EINVAL; // 如果传入长度不合法, 返回无效参数错误码
    }
    temp_buf = kmalloc(max_length, GFP_KERNEL); // 在内核空间分配临时缓冲区
    if (!temp_buf) {
        return -ENOMEM; // 如果内存分配失败, 返回内存不足错误码
    }
    if (copy_from_user(temp_buf, input_str, max_length)) {
        kfree(temp_buf); // 如果从用户空间拷贝数据失败, 释放临时缓冲区并返回错误
        return -EFAULT;
    }
    for (i = 0; i < max_length && temp_buf[i] != '\0'; i++) {
        if (temp_buf[i] >= 'a' && temp_buf[i] <= 'z') {
            temp_buf[i] -= 32; // 将小写字母转换为大写字母
        }
    }
    if (copy_to_user(output_str, temp_buf, max_length)) {
        kfree(temp_buf); // 如果向用户空间拷贝数据失败, 释放临时缓冲区并返回错误
        return -EFAULT;
    }
    kfree(temp_buf); // 完成操作后释放临时缓冲区
    return i; // 返回实际处理的字符串长度 (不包含结束符)
}
```

3.3 内核重编译

3.3.1 内核重编译步骤介绍

完成系统调用函数的编写和系统调用表的更新后, 我们需要对 Linux 内核进行编译, 以生成包含我们新增系统调用的新内核。首先, 进入 `/usr/src/linux` 目录, 这是 Linux 内核源代码的根目录:

```
cd /usr/src/linux

make -j 2
```

这个命令会启动内核的编译过程。其中, `-j 2` 选项表示使用两个并行的编译任务来加速编译过程, 这个数字可以根据你的计算机的 CPU 核心数进行调整。(使用 `make -j 2` 实现了以下两步:

```
make bzImage    #编译映像文件
```

```
make modules    #编译模块)
```

编译完成后，我们需要安装新编译的内核模块。在/usr/src/linux目录下执行以下命令：

```
make modules_install
```

这个命令会将新编译的内核模块安装到系统中，使得它们能够在新内核启动时被加载。

3.3.2 新内核重启前的准备

安装新内核是整个过程中的最后一步，也是至关重要的一步。我们需要将编译生成的新内核及相关文件复制到系统的启动目录中，并更新启动配置，以确保计算机能够从新内核启动。

首先，将编译生成的 System.map 文件复制到 /boot/System.map-2.6.33.3。System.map 文件包含了内核的符号表，它对于内核调试和分析非常重要。

拷贝新内核到引导区

```
cp    /usr/src/linux/System.map    /boot/System.map-2.6.33.3
```

接着，将编译生成的 bzImage 文件复制到 /boot/vmlinuz-2.6.33.3。bzImage 是压缩的内核映像文件，它是启动 Linux 系统的核心文件。

```
cp    /usr/src/linux/arch/i386/boot/bzImage
```

```
/boot/vmlinuz-2.6.33.3
```

然后，将内核配置文件.config 复制到/boot/config-2.6.33.3。这个文件记录了内核的配置选项，对于后续的内核维护和升级具有参考价值。

```
cp /usr/src/linux/.config /boot/config-2.6.33.3
```

接下来，我们需要生成新的初始内存文件系统 initrd.img。初始内存文件系统是一个临时的文件系统，它在系统启动的早期阶段被加载到内存中，用于提供必要的驱动程序和脚本，以便系统能够访问硬盘上的根文件系统。使用以下命令生成新的 initrd.img

建立要载入的 ramdisk 映像文件

```
mkinitramfs -o /boot/initrd.img-2.6.33.3 2.6.33.3
```

最后，我们需要更新启动配置，使得计算机能够从新安装的内核启动。进入/boot 目录，移除旧的 System.map 和 vmlinuz 文件，然后创建新的符号链接，将 vmlinuz-2.6.33.3 链接为 vmlinuz，将 System.map-2.6.33.3 链接为 System.map。

重新创建链接

```
cd /boot
rm -f System.map vmlinuz
ln -s vmlinuz-2.6.33.3 vmlinuz
ln -s System.map-2.6.33.3 System.map
```

3.3.3 重启新内核

完成以上步骤后，确认内核版本如图所示（版本为 2.6.33.3）。
重启计算机，系统将从新安装的内核启动，我们的新增系统调用也将随之生效。



4 验证 **sytest** () 系统调用

4.1 编译验证程序 **test.c**

编写测试代码 2.c

```
include <stdio.h>
include <stdlib.h>
include <sys/syscall.h>
include <unistd.h>
include <string.h>
define __NR_str_process 339

nt main()

char input_str[100]; // 定义字符数组用于存储用户输入的字符串，长度可根据需求调整
char output_str[100]; // 定义用于存储处理后字符串的字符数组
int max_length;
int result;

// 提示用户输入字符串
printf("请输入一个字符串，然后按下回车键：\n");
fgets(input_str, sizeof(input_str), stdin); // 从标准输入读取用户输入的字符串，最多读取sizeof(input_str) - 1个字
max_length = strlen(input_str);
if (input_str[max_length - 1] == '\n') { // 如果最后一个字符是换行符（按下回车键导致）
    input_str[max_length - 1] = '\0'; // 将换行符替换为字符串结束符，使字符串格式正确
    max_length--; // 长度减1，去除换行符的长度
}

// 发起系统调用，传入输入字符串地址、输出字符串地址以及最大长度
result = syscall(__NR_str_process, input_str, output_str, max_length);
if (result == -1) {
    perror("System call failed");
    return -1;
}
printf("The processed string is: %s\n", output_str);
return 0;
```

```
root@student-desktop:~# gedit 2.c
root@student-desktop:~# gcc 2.c -o 2
```

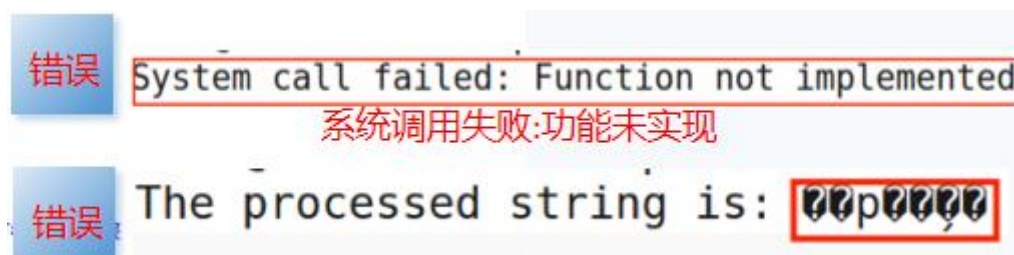
4.2 运行验证程序 **test**

调入成功：

调用
成功

```
root@student-desktop:~# ./2
请输入一个字符串，然后按下回车键：
poiuytf
The processed string is: P0IUyTF
```

调入失败：



在完成了系统调用的添加和内核的安装后，对新增的系统调用进行验证，确保正常工作。然而，在实操中，我们遇到了问题。在初次验证时，系统调用失败，提示“功能未实现”。原因如下：

1. **系统调用表更新不完整**：在添加系统调用到系统调用表时，可能没有正确地将系统调用函数与系统调用号关联起来，或者在更新系统调用表后没有正确地编译内核，导致新系统调用没有被正确地加载到内核中。

2. **内核配置问题**：在编译内核时，可能存在配置选项不正确的问题，导致新增的系统调用无法正常工作。例如，如果内核配置中没有启用某些必要的模块或功能，可能会导致系统调用无法找到所需的资源或服务。

3. **用户程序调用方式错误**：在用户程序中调用系统调用时，可能没有正确地使用系统调用号或参数，导致系统调用无法被正确地识别和执行。

为了解决这些问题，我们需要仔细检查系统调用表的更新是否完整，确认内核配置是否正确，并检查用户程序。

5 课程设计的体会

5.1 个人工作体会

在本次课程设计中，我主要承担了内核重编译和在 Linux 内核源代码中添加系统调用这一关键任务，这一过程犹如一场充满挑战与收获的探索之旅。

5.1.1 系统调用的设计与实现

(1) 功能设计的考量

在着手设计系统调用功能时，我将实用性和简洁性作为首要目标，经过深思熟虑后确定了 `sys_str_process` 这一功能，专注于字符串大小写转换。为了实现良好的交互效果，我精心规划了参数传递与返回值设计。

对于输入参数，`input_str` 的设定旨在接收用户输入的字符串，为系统调用提供待处理的数据来源。`output_str` 则用于存放转换后的结果，确保转换后的字符串能够准确地传递回用户空间。而 `max_length` 参数的引入，主要是为了限制字符串长度，避免因过长的字符串导致内存分配异常或其他潜在问题。这种参数设计方式，使得系统调用在处理字符串时具有明确的边界和规则，增强了其稳定性和可靠性。

在返回值设计方面，选择返回实际处理的字符串长度（不包含结束符），这一设计为用户提供了直观且有用的信息。用户可以通过返回值了解系统调用对字符串的处理程度，从而更好地判断结果的有效性，方便在后续程序中进行相应的处理和决策。

(2) 函数实现的探索

当开始实现函数实体时，我仿佛踏入了 Linux 内核的神秘领域，每一行代码都如同在未知的道路上前行，需要格外谨慎。

在使用 `kmalloc` 函数分配内核空间临时缓冲区时，我深入研究了其参数的意义。其中，参数 `max_length` 决定了分配内存的大小，确保能够容纳用户输入的字符串。而 `GFP_KERNEL` 标志则明确了内存分配的行为模式，它表示在常规内核路径中进行内存分配，可能会睡眠等待内存资源。通过对这些参数的深入理解，我能够更加精准地控制内存分配过程，确保分配的内存既满足需求又不会造成资源浪费，从而实现高效的内存管理。

在字符串转换逻辑部分，通过字符的 `ASCII` 码范围判断大小写并进行转换是关键步骤。我清晰地认识到，在 `ASCII` 码表中，小写字母的范围是'a'到'z'，其对应的大写字母 `ASCII` 码值比小写字母小 32。基于这一特性，我编写了转换逻辑，当字符在小写字母范围内时，通过减去 32 的方式将其转换为大写字母。这一过程不仅实现了字符串大小写的转换功能，更让我对字符编码在计算机底层的表示方式有了深刻的理解，明白了字符在计算机中是以数字形式

存储和处理的，而这种数字与字符之间的映射关系是计算机处理文本信息的基础。

在与内核空间 and 用户空间进行数据交互时，`copy_from_user` 和 `copy_to_user` 函数的使用让我深刻体会到了内核与用户空间隔离及数据传递的重要性与复杂性。`copy_from_user` 函数用于将用户空间的数据拷贝到内核空间，在这个过程中，我需要确保传递的参数正确无误，特别是源地址（用户空间字符串地址）和目标地址（内核空间临时缓冲区地址）以及拷贝长度的准确性。如果这些参数出现错误，可能会导致数据拷贝失败，甚至引发系统异常。同样，`copy_to_user` 函数用于将内核空间处理后的结果拷贝回用户空间，这也需要严格遵循函数的参数要求，保证数据能够正确地传递给用户程序。这些函数的使用让我明白，内核与用户空间之间的数据交互是一个严谨而复杂的过程，任何小的疏忽都可能导致严重的后果，从而进一步强化了我对系统稳定性和安全性的重视。

5.1.2 踩坑与解决过程

(1) 系统调用表更新与链接错误:

在修改系统调用表项时，问题接踵而至。按照文档在 `syscall_table_32.S` 文件末尾添加 `long sys_str_process` 后，编译内核却出现链接错误。起初，我陷入困惑，反复检查添加的代码却毫无头绪。经过冷静分析，我决定深入研究系统调用号相关机制，这才发现是 `unistd.h` 文件中系统调用号分配的问题。原来，新定义的系统调用号

与现有系统调用号冲突，导致链接时无法正确定位函数入口。为解决此问题，我重新梳理系统调用号分配规则，仔细查找未被使用的号段，最终确定 339 为合适的系统调用号，并确保在 `unistd.h` 文件中正确定义和关联。

(2) 数据拷贝与参数合法性问题:

实现系统调用函数实体时，`copy_from_user` 函数的错误返回值让我头疼不已。程序总是在数据拷贝环节出错，导致无法正常运行。我开始大量查阅 Linux 内核编程资料，逐步排查可能的原因。经过长时间的调试与分析，我发现问题在于 `max_length` 参数的处理。当传入的长度超过用户空间实际字符串长度时，拷贝操作会失败。于是，我在函数开头添加了严格的 `max_length` 合法性检查，确保传入参数在合理范围内，避免了后续一系列错误。

(3) 内核配置与功能实现问题:

在完成系统调用添加并进行初步测试时，遇到了系统调用失败提示“功能未实现”的问题。我首先检查了系统调用表的更新，确认系统调用号与函数的关联正确无误，并且重新编译了内核，但问题依旧存在。接着，我怀疑是内核配置的问题，仔细检查了编译内核时的各项配置选项，发现可能是某些与字符串处理相关的模块没有正确启用。经过深入研究内核配置文档，重新调整了相关配置参数，确保启用了必要的模块和功能。然而，问题仍然没有得到解决。最后，我将

目光聚焦到用户程序调用方式上，发现测试程序中对系统调用号的定义与内核中不一致，导致系统无法识别系统调用。修正这个错误后，系统调用终于成功执行，输出了正确的结果。

5.1.3 心得体会

此次经历让我对 Linux 内核的理解达到了一个新的高度。以往对内核的认识仅停留在理论知识层面，通过实际动手修改源代码，我深入了解了系统调用从发起请求到内核处理再返回结果的全过程。这使我明白，操作系统内核如同一个精密复杂的机器，每个部分都紧密协作，任何细微的改动都可能影响整体性能。这种深入理解不仅丰富了我的知识体系，更让我在系统级编程思维上有了质的飞跃，为未来从事相关工作积累了宝贵经验。

在解决问题的过程中，我的问题解决能力得到了全方位的锻炼。从最初面对错误的无助与迷茫，到逐渐学会冷静分析错误信息，运用所学知识和查阅资料寻找线索，再到最终确定解决方案，每一步都是成长的见证。我深刻体会到，技术问题的解决需要耐心、细心与恒心。每一个看似无解的问题，只要坚持不懈地深入研究，总能找到突破的关键。这种能力的提升将使我在未来面对各种技术挑战时更加从容自信。

同时，这次课程设计也让我深刻认识到坚持和耐心的力量。在漫长的内核修改与问题解决过程中，时间和精力投入常常超出预期。

但正是凭借着坚持到底的决心和足够的耐心，我才能逐步克服一个又一个难题。每一次问题的解决都像是攻克了一座堡垒，让我更加坚信，只要保持积极的态度和坚韧不拔的精神，任何困难都能被战胜。这种心态的转变将对我今后的学习和工作产生深远的影响，激励我在面对复杂任务时勇往直前。

5.2 团队合作

在本次课程设计中，团队合作是项目成功的坚实保障，尽管我专注于内核添加系统调用工作，但团队成员间的紧密协作贯穿始终。

5.2.1 知识共享与交流

团队成员之间的知识共享如同明亮的灯塔，照亮了我在探索内核编程之路上的迷茫。在遇到系统调用号冲突问题时，大家共同查阅资料，热烈讨论系统调用号的分配规则。有的成员分享了他们在其他类似项目中遇到的系统调用号管理经验，有的则从内核源码的整体架构角度提出了可能的解决方案。通过这种开放的交流，我不仅顺利解决了眼前的问题，还对系统调用号的分配原则有了更全面、深入的理解。

我也积极将自己在实现系统调用函数过程中积累的内核内存管理和字符串处理经验分享给团队。在团队定期的技术交流会上，我详细讲解了 `kmalloc` 函数的使用技巧以及字符串转换逻辑中的注意事项。成员们的积极反馈和提问让我对这些知识有了更深层次的思考，

同时也促进了团队整体技术水平的提升。这种知识的双向流动使我们每个人都能在团队中汲取营养，共同成长。

5.2.2 协同工作与问题解决

在解决涉及多环节的复杂问题时，团队的协同作战能力发挥得淋漓尽致。例如，在测试新系统调用时，用户程序调用失败。负责测试的成员迅速反馈问题，我们立即展开联合调试。我从内核代码角度检查系统调用的实现是否存在漏洞，测试成员则仔细分析用户程序的参数传递和调用逻辑。通过共享调试信息和不断尝试不同的测试场景，我们最终发现是用户程序对字符串结尾符处理不当导致内核无法正确识别长度。在这个过程中，团队成员间的默契配合和高效沟通是问题得以解决的关键。

另外，在配置编译环境和解决依赖关系问题时，团队成员分工合作。有的成员负责研究编译参数的含义与优化方法，有的成员则专注于解决依赖库的安装与版本兼容性问题。我们定期汇总各自的进展情况，共同商讨解决方案，确保了编译环境的稳定和正确配置。这种协同工作模式不仅提高了解决问题的效率，还增强了团队成员之间的信任与合作能力。

5.2.3 团队精神的激励

团队精神如同温暖的阳光，在困难时刻给予我力量。在面对内核修改过程中接连不断的问题时，我曾多次感到沮丧和疲惫。然而，团队成员们始终相互鼓励、相互支持。每当我陷入困境时，他们会主动

分享自己在解决类似问题时的经历，给予我信心和启发。一句简单的“加油，我们一起解决”都能让我重新振作起来，继续投入到紧张的工作中。

团队共同庆祝每一个小成就的时刻也让我深刻体会到团队精神的魅力。每当成功解决一个关键问题或完成一个重要阶段的任务时，我们会一起回顾走过的艰辛历程，分享喜悦之情。这种积极向上的氛围不仅增强了团队的凝聚力，也让我更加珍惜团队合作的时光，激励着我为了团队的共同目标而努力奋斗。在未来的学习和工作中，我将永远铭记团队合作的力量，并积极融入团队，共同追求卓越。